

Übung 2: Methoden (Obligatorische Abgabe!)

Aufgabe 0: Parameterübergabe	1
Aufgabe 1: Array-Funktionen	2
Aufgabe 2: Rekursive Doppelfakultät	3
Aufgabe 3: Rekursive Summe	3
Aufgabe 4: Sieb des Eratosthenes	3
Aufgabe 5: Typ-Konversion (Theorie)	4
Aufgabe 6: Numerische Erweiterung (Theorie)	5

Lernziele

- Methoden entwerfen und verwenden.
- Rekursion nutzen und verstehen.
- Primitive Datentypen vertiefen.

Hinweis:

- **Beachten Sie, dass Sie diese Übung abgeben müssen, um zur Prüfung zugelassen zu werden! Sie finden die Frist für Ihre Übungsgruppe in Moodle.**
- Laden Sie die Java-Dateien mit Ihren Lösungen rechtzeitig in der dafür vorgesehenen Abgabe auf Moodle hoch.
- **Die Theorieaufgaben (5 und 6) müssen Sie nicht abgeben.**

Aufgabe 0: Parameterübergabe

In der Vorlesung haben wir die beiden Methoden `symbolSquare()` und `symbolLine()` angeschaut:

```
1  void symbolSquare(char symbol, int amount) {
2      for (int i = 1; i <= amount; i++) {
3          symbolLine(symbol, amount);
4      }
5  }
6
7  void symbolLine(char symbol, int amount) {
8      for (int i = 1; i <= amount; i++) {
9          IO.print(symbol);
10     }
11     IO.println();
12 }
```

Sie möchten den Code nun anpassen, dass kein Quadrat, sondern stattdessen ein Dreieck ausgegeben wird:

```

1  $$$$ $$$$
2  $$$$ $$$$
3  $$$$      -> $$$
4  $$$$      $$$
5  $$$$      $$
                $
  
```

Nach jeder vollständigen Zeile soll also der `amount` an `$` um eins reduziert werden. Reicht dazu die folgende Anpassung auf Zeile 6?

```

1  void symbolLine(char symbol, int amount) {
2      for (int i = 1; i <= amount; i++) {
3          IO.print(symbol);
4      }
5      IO.println();
6      amount--; // reicht diese Änderung?
7  }
  
```

Aufgabe 1: Array-Funktionen

Für Arrays sollen folgende nützliche Funktionalitäten als Methoden entwickelt werden:

Name	Eingabe	Rückgabe	Beschreibung
<code>revert()</code>	int -Array (int [])	int -Array (int [])	Retourniert Array mit umgekehrter Reihenfolge der Elemente wie die Eingabe. Z.B. [1, 2, 3] wird [3, 2, 1].
<code>isAscending-lySorted()</code>	int -Array (int [])	boolean	Prüft, ob die Elemente der Eingabe alle aufsteigend sortiert sind. Z.B. [-1, 0, 7] ergibt true .
<code>isDescending-lySorted()</code>	int -Array (int [])	boolean	Prüft, ob das Array absteigend sortiert ist.
<code>print()</code>	int -Array (int [])	(keine)	Gibt alle Elemente des Arrays auf der Konsole aus. Beispielausgabe: [1, 2, 3].

Entwerfen und implementieren Sie die entsprechenden Methoden. Zwei Testklassen sind bereits vorgegeben, führen aber im Moment noch zu Fehlern, da die von den Tests aufgerufenen Methoden noch nicht implementiert sind.

Aufgabe 2: Rekursive Doppelfakultät

Implementieren Sie die Berechnung der Doppelfakultät (siehe auch letzte Woche) dieses Mal jedoch als rekursive Funktion. Die Methode soll nun mit dem Datentyp **long** arbeiten, um grössere Ergebnisse zu unterstützen.

Zur Erinnerung: Die Doppelfakultät von n ist definiert als:

$$f(n) = \begin{cases} 1 * 3 * 5 * \dots * (n-2) * n & \text{für ungerade } n \geq 1 \\ 2 * 4 * 6 * \dots * (n-2) * n & \text{für gerade } n \geq 2 \end{cases}$$

Überprüfen Sie die entwickelte Funktionalität mit ein paar Eingaben. Was geschieht bei sehr grossen Werten für n , etwa 100'000 und mehr?

Aufgabe 3: Rekursive Summe

Gegeben sei ein **int**[]. Berechnen Sie die Summe der dort enthaltenen Werte mithilfe von Rekursion.

Hinweise:

- Entwerfen Sie eine Methode, welche die Summe für den Teilarray ab einer bestimmten Position berechnet.
- Alternativ kann man auch die Summe für ein leeres Array bestimmen und dann das Array immer entsprechend kürzen.
- In der Vorlage sind Werte vorgegeben. Die Gesamtsumme sollte für diese Vorlage 47'709 ergeben.

Aufgabe 4: Sieb des Eratosthenes

Das Sieb des Eratosthenes haben wir letzte Woche bereits angeschaut und auch eine Implementierung zur Berechnung erstellt.

Zur Erinnerung: Der Algorithmus folgt dabei diesem Ablauf:

1. Ein Array mit allen Zahlen bis zur höchsten Zahl erstellen.
2. Für alle Zahlen im Array ab der 2: alle Vielfachen der Zahl im Array streichen.
3. Schritt 2 für die übrigbleibenden Zahlen einzeln wiederholen.
4. Die noch übriggebliebenen Zahlen ausgeben.

In der Vorlage finden Sie die Musterlösung vom letzten Mal. Nun wollen wir den Code aber noch verbessern, indem wir gute Praktiken des OO-Designs einsetzen, etwa eine sinnvolle funktionale Zerlegung und passend gewählte Namen für Variablen und Methoden:

- Die `main()`-Methode soll möglichst viele Details der Implementierung in zusätzliche Methoden auslagern, so dass `main` nur noch den groben Ablauf wie im Ablauf oben enthält.
- Um gestrichene Einträge im Array zu markieren, verwenden wir eine negative Zahl. Alternativ könnte man auch `fix` den Wert `-1` nutzen. Ein solcher Wert hat also nun eine spezielle Bedeutung, man spricht auch von einer «Magic Number». Solche Zahlen sollten immer als (konstante – dazu kommen wir noch) Variable definiert werden.
- Die Ausgabe auf der Konsole ist noch nicht besonders schön, da am Ende immer noch ein Komma steht: `..., 67, 71, 73, 79, 83, 89, 97`, Verbessern Sie die Ausgabe möglichst auch noch.

Aufgabe 5: Typ-Konversion (Theorie)

Gegeben seien diese Definitionen von Variablen:

```
1 char c = '!';  
2 byte b = 0;  
3 short s = 1;  
4 int i = 3;  
5 long l = 4;  
6 float f = .5f;  
7 double d = 0.6;
```

Bei welchen Anweisungen in der untenstehenden Sequenz müssen noch Type Casts angegeben werden? Analysieren Sie es zuerst ohne Computer und verifizieren Sie Ihre Annahme danach. Vermeiden Sie überflüssige Type Casts.

- 1) `long z = b;`
- 2) `int a = f * 2;`
- 3) `float x = 0x12345678;`
- 4) `double y = l * l;`
- 5) `z = y * y;`
- 6) `s = b + 1;`
- 7) `b = c + 1;`
- 8) `b += 1;`

Aufgabe 6: Numerische Erweiterung (Theorie)

Bestimmen Sie das Resultat folgender Zuweisungen mit einem Java-Programm und versuchen Sie nachzuvollziehen, wieso es diese Resultate gibt.

Hinweis: `Integer.MAX_VALUE` == 2147483647 ($2^{31} - 1$).

```
1 int i = 1;  
2 long l = 2;  
3 float f = .1f;  
4 double d = 0.2;
```

- 1) `float` x1 = i / 4;
- 2) `double` x2 = i / 4.0;
- 3) `float` x3 = 1.0f / l;
- 4) `double` y1 = 123456789 + f;
- 5) `double` y2 = 123456789.0 + f;
- 6) `long` z1 = `Integer.MAX_VALUE` + i;
- 7) `long` z2 = `Integer.MAX_VALUE` * 2;
- 8) `float` z3 = i / 0;